Edge–Fog–Cloud Task Placement Simulator

Course: ARM 305 — Cloud, Dev, Edge and Fog Computing
Repository: NoticeableStar/edge-fog-cloud-simulator

Abstract

This project implements a Python simulator for task placement across an Edge–Fog–Cloud computing hierarchy. It models heterogeneous user tasks with different CPU and memory requirements, latency sensitivities, and tenant identities. The simulator compares two placement strategies: Latency-First, which selects tiers according to task sensitivity, and Capacity-Aware, which balances remaining capacity against latency. The project evaluates policy performance, overload behavior, tenant isolation, and the effect of upgrading Edge capacity. In the 12-task policy comparison, both policies completed all tasks. Latency-First achieved an average modeled latency of 80.00 ms, compared with 91.67 ms for Capacity-Aware. The overload test admitted 13 of 19 tasks and rejected six. An Edge upgrade reduced the reported average latency from 80.00 ms to 75.42 ms. These results demonstrate the simulator's ability to compare placement decisions and resource use under defined assumptions. They are simulation results, not measurements from deployed hardware.

1. Introduction and Objective

Edge computing places processing near users and devices to reduce communication delay. Fog computing offers an intermediate tier, while Cloud data centers provide larger resource pools at the cost of higher modeled latency. Choosing where a task should run therefore involves a trade-off between response time and available capacity.

The objective of this project is to build a repeatable simulation environment that:

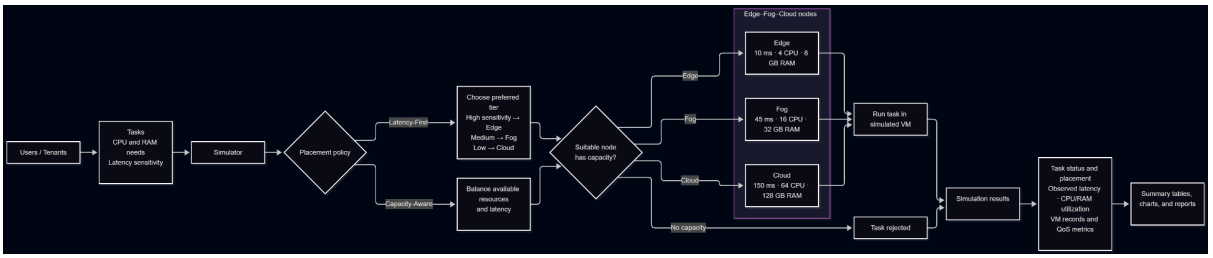
1. Represents Edge, Fog, and Cloud nodes with different capacities and latency values.
2. Processes a multi-tenant workload containing tasks with varied resource requirements and latency sensitivities.
3. Compares latency-oriented and capacity-oriented placement policies.

4. Records placements, resource use, task outcomes, and quality-of-service (QoS) metrics.
5. Tests overload handling, tenant access controls, and the effect of Edge scaling.

The project is intended for demonstration and comparative experiments. It does not provision real virtual machines or measure production network latency.

2. System Architecture

The simulator accepts a workload of tasks from multiple tenants. For each task, a placement policy selects an eligible node, or the task is rejected if no node has sufficient CPU and RAM. A simulated VM record is created for each admitted task. The simulator then reports placement, resource utilization, and QoS results.



The default topology and node properties are defined in [core/simulator.py](). Each tier is represented by one node:

Tier	Base Latency	CPU Capacity	RAM Capacity
Edge	10 ms	4 cores	8 GB
Fog	45 ms	16 cores	32 GB
Cloud	150 ms	64 cores	128 GB

The workload generator provides 12 benchmark tasks from traffic, healthcare, and analytics tenants, with varying CPU and RAM requests and high, medium, or low latency sensitivity. The overload workload adds seven high-demand tasks to the benchmark workload.

3. Placement Algorithms

The two placement strategies are implemented in [core/placement.py.]()

3.1 Latency-First

Latency-First orders node tiers based on task sensitivity, then checks them in order until it finds one with enough CPU and RAM:

- High sensitivity: Edge → Fog → Cloud
- Medium sensitivity: Fog → Edge → Cloud
- Low sensitivity: Cloud → Fog → Edge

If no node in the preference order can host the task, placement fails and the task is recorded as rejected. This approach gives task sensitivity a clear influence on placement, while still allowing fallback when a preferred tier runs out of capacity.

3.2 Capacity-Aware

Capacity-Aware first filters out nodes that cannot host the task. For each eligible node, it calculates CPU and RAM headroom and combines this with a normalized latency penalty. The score used by the implementation is:

$$\text{score} = (1-w) \times \text{headroom} - w \times \text{normalized latency}$$

Here, `headroom` is the average of the node's remaining CPU and RAM fractions. The sensitivity weight w is 0.6 for high-sensitivity tasks, **0.3** for medium, and **0.1** for low. The policy chooses the eligible node with the highest score. This strategy attempts to avoid concentrating all work on the lowest-latency tier. Its trade-off is that a less-loaded, higher-latency node may be selected over a closer node.

4. Implementation

The simulator is organized into a core engine, workload and experiment modules, and user-facing interfaces:

- **`core/models.py`:** Data models for tasks, compute nodes, virtual-machine records, and task/VM statuses.
- **`core/placement.py`:** Latency-First and Capacity-Aware placement policies.
- **`core/simulator.py`:** Workload execution, resource allocation, task outcomes, and QoS metric calculation.
- **`core/security.py`:** Role-based access control, tenant isolation, and audit logging.
- **`experiments/default_tasks.py`:** Benchmark and overload task definitions.

- **`experiments/runner.py`:** Runs the four experiments and exports result tables and charts.
- **`main.py` and `app.py`:** Command-line and Streamlit interfaces.

For each admitted task, the simulator creates a VM record, allocates the requested CPU and RAM on the selected node, and records the assigned node and modeled latency. If no eligible node is available, the task is marked rejected with an insufficient-capacity reason. The observed task latency is set to the selected node's configured base latency; it is not derived from real network or execution measurements.

The experiment runner uses pandas for tabular data and Matplotlib for charts. It exports CSV and JSON files to outputs/tables/ and PNG charts to outputs/charts/.

5. Experiments and Results

The experiment runner defines four tests: policy comparison, overload and rejection, unauthorized access, and Edge parameter tuning. Results below are from the repository's generated outputs.

5.1 Policy Comparison

The same 12-task workload was run with each policy. Both policies admitted all 12 tasks. Task placement counts, QoS metrics, and resource utilization are summarized below.

Placement & QoS Performance

Placement Policy	Placement (Edge / Fog / Cloud)	Avg. Latency	Success Rate
Latency-First	3 / 4 / 5 tasks	**80.00 ms**	100% (12/12)
Capacity-Aware	2 / 4 / 6 tasks	91.67 ms	100% (12/12)

Resource Utilization by Tier

Placement Policy	Edge Utilization	Fog Utilization	Cloud Utilization
Latency-First	68.75%	75.00%	56.25%
Capacity-Aware	56.25%	65.62%	59.38%

Latency-First had the lower average modeled latency in this workload. Capacity-Aware placed one more task on Cloud and showed lower Edge and Fog utilization, distributing demand more evenly across tiers. The comparison illustrates the trade-off between latency and load distribution rather than proving that one policy is universally superior.

5.2 Overload and Rejection

The overload test submitted **19 tasks** to the cluster, with total requested CPU exceeding the available capacity. The simulator admitted 13 tasks and rejected 6, giving a success rate of **68.42%**. The test demonstrates that tasks are rejected when no candidate node has sufficient CPU and RAM, rather than being assigned beyond modeled capacity.

5.3 Multi-Tenant Security

The security experiment authenticated a user belonging to tenant_traffic and attempted to access records for `tenant_health`. The request was blocked with a **403 Forbidden** error and recorded in the security audit log. This demonstrates the simulator's tenant-isolation behavior for the tested access scenario.

5.4 Edge Scaling

The baseline Edge node (4 CPU, 8 GB RAM, 10 ms) was compared with an upgraded Edge node (8 CPU, 16 GB RAM, 5 ms), using the same workload and Latency-First policy.

Edge Scaling Performance & Allocation

Configuration	Edge VMs	Avg. Latency	Latency Delta
Baseline Edge	3	80.00 ms	-
Upgraded Edge	4	75.42 ms	-4.58 ms

Tier Utilization Comparison

Configuration	Edge Utilization	Fog Utilization	Cloud Utilization
Baseline Edge	68.75%	75.00%	56.25%
Upgraded Edge	59.38%	62.50%	56.25%

The upgraded Edge accommodated one additional VM, and the average modeled latency fell by **4.58 ms**. Utilization values are the reported post-run metrics; the increased node capacity also changes the utilization denominator.

6. Limitations

The results should be interpreted within the simulator's assumptions:

1. **Modeled rather than measured performance:** Node latencies are fixed values, and observed task latency is assigned from the selected node's base latency. Real execution time, network congestion, and communication overhead are not measured.
2. **Small, fixed topology:** The default cluster contains one node per tier. It does not evaluate multiple nodes, dynamic node availability, or geographically distributed sites.
3. **Preset workloads:** The experiments use a small set of predefined tasks. Real workloads may have different arrival patterns, task dependencies, resource demands, and changing latency requirements.
4. **Simplified resource model:** Tasks reserve CPU and RAM, but the model does not simulate contention effects, storage, bandwidth, energy use, or execution duration.
5. **Policy-specific comparison:** The Capacity-Aware score uses fixed sensitivity weights and a particular headroom calculation. Its performance may differ with other workloads or policy parameters.
6. **Limited security evaluation:** The security test demonstrates a specific tenant-access denial scenario; it is not a comprehensive security assessment.

These limitations mean that the findings support educational comparison of placement strategies, not direct predictions of production-system performance.

7. Conclusion and Reproduction

The project provides a working simulation of task placement across Edge, Fog, and Cloud tiers. It compares a sensitivity-driven policy with a capacity-aware scoring policy and records placement, resource utilization, task success or rejection, and modeled latency. In the tested benchmark, Latency-First achieved lower average latency, while Capacity-Aware distributed load more evenly. Additional experiments demonstrated overload rejection, tenant access denial, and a modest latency improvement after increasing Edge capacity.

To regenerate experiment outputs from the repository, install the project dependencies and run:

```
None
python -m experiments.runner
```

The generated charts and tables are saved under `outputs/charts/` and outputs/tables/. The experiment definitions and export logic are in `experiments/runner.py`; the generated summary is `outputs/tables/all_experiments_summary.json`.